```
                    f"""
                    INSERT INTO flight_latest ({", ".join(flight_cols)})
                    VALUES ({", ".join(["?"] * len(flight_cols))})
                    ON CONFLICT(flight_key) DO UPDATE SET
                        {", ".join([f"{c}=excluded.{c}" for c in update_cols])}
                    """,
                    tuple(flight_vals),
                )
                self._last_ingest_wallclock[flight_key] = ingest_wallclock

                if q["lat_e6"] == 0 and q["lon_e6"] == 0:
                    continue
                if not self._should_store(flight_key, ts_iso, q):
                    continue

                traj_cols = ["flight_key", "timestamp", "lat_e6", "lon_e6"]
                traj_vals = [flight_key, ts_iso, q["lat_e6"], q["lon_e6"]]
                if include_traj_lat_e5:
                    traj_cols.append("lat_e5")
                    traj_vals.append(lat_e5)
                if include_traj_lon_e5:
                    traj_cols.append("lon_e5")
                    traj_vals.append(lon_e5)
                traj_cols.extend(["altitude_m", "speed_cms", "heading_de2", "vs_cms", "squawk", "source_ip"])
                traj_vals.extend([
                    q["altitude_m"],
                    q["speed_cms"],
                    q["heading_de2"],
                    q["vs_cms"],
                    q["squawk"] if q["squawk"] > 0 else None,
                    source_ip,
                ])
                cur.execute(
                    f"""
                    INSERT OR IGNORE INTO trajectory_samples ({", ".join(traj_cols)})
                    VALUES ({", ".join(["?"] * len(traj_cols))})
                    """,
                    tuple(traj_vals),
                )
                self._sample_counter[flight_key] = self._sample_counter.get(flight_key, 0) + 1
                self._last_saved[flight_key] = {
                    "dt": ts_dt,
                    "lat_e6": q["lat_e6"],
                    "lon_e6": q["lon_e6"],
                    "altitude_m": q["altitude_m"],
                    "speed_cms": q["speed_cms"],
                    "heading_de2": q["heading_de2"],
                }
                appended_rows.append(
                    {
                        "flightKey": flight_key,
                        "timestamp": ts_iso,
                        "icaoAddress": icao or "UNKNOWN",
                        "callsign": callsign or "UNKNOWN",
                        "latDD": q["lat_e6"] / 1000000.0,
                        "lonDD": q["lon_e6"] / 1000000.0,
                        "altitudeM": q["altitude_m"],
                        "speedCMS": q["speed_cms"],
                        "headingDE2": q["heading_de2"],
                        "verVelocityCMS": q["vs_cms"],
                        "squawk": q["squawk"] if q["squawk"] > 0 else None,
                        "sourceIp": source_ip,
                    }
                )

            self._conn.commit()
            if self.append_csv_enabled and appended_rows:
                self._append_trajectory_csv_rows(appended_rows)
            if self.icao_excel_enabled and rows:
                self._append_icao_excel_rows(rows)
            self._purge_old_history_if_needed()

    def get_latest_matrix(self, limit=1000):
```